

4. El Departament de Salut está buscando un sistema inteligente que ayude a gestionar el envío inmediato de material bio-sanitario dentro de una área sanitaria. Para ello ha decidido crear una pequeña flota de vehículos eléctricos compuesta por mini-vans y motos eléctricas. A través de un servicio web diferentes entidades (hospitales, laboratorios de análisis, residencias de ancianos) podrán solicitar el envío de paquetes con material bio-sanitario: guantes, máscaras, equipos de protección individual (EPIs), gel hidroalcohólico, jabon desinfectante, toallas secantes de papel, pruebas PCR, muestras de sangre y material de desecho a incinerar.

Un planificador recibe las peticiones recibidas en los últimos 20 minutos (cada una de ellas incluye el material requerido, el lugar donde se ha de recoger el material y el lugar de destino en el que se ha de entregar el material) y crea un plan para servir las peticiones con los vehículos disponibles. Los vehículos salen de un parking municipal y después de hacer todos los transportes requeridos han de volver al parking para recargar sus baterías. Todos los paquetes son de una medida estandar, y los vehículos han sido preparados con contenedores especiales para transportar paquetes de esa medida. En el caso de las mini-vans disponen de entre 2 y 6 contenedores especiales refrigerados, mientras que las motos eléctricas disponen de un único contenedor especial que no está refrigerado. Hay materiales (como los PCRs y las muestras de Sangre) que solo se pueden transportar dentro de un contenedor refrigerado. Pero para aprovechar los viajes se permite a las mini-vans que lleven también materiales que no precisen refrigeración en sus contenedores refrigerados. Cada vehículo solo puede llevar un paquete de material por contenedor, y no puede coger más paquetes si tiene todos sus contenedores llenos.

El plan generado ha de mostrar claramente como se mueve cada vehículo de un lugar a otro, y lo que van recogiendo y entregando en cada lugar.

- a) Describe el dominio (incluyendo predicados, acciones, etc...) usando PDDL. Da una explicación razonada de los elementos que has escogido. Explica el significado de cada predicado y sus parámetros (por ejemplo: "*encima ?X ?Y* = el elemento *?X* está encima del elemento *?Y*"). Explica también si añades predicados para filtrar posibles asignaciones o para guiar mejor al planificador hacia el objetivo. Ten en cuenta que el modelo del dominio ha de poderse extender a más o menos paquetes, lugares de recogida y entrega y vehículos (con más o menos contenedores de transporte).

Existen muchas posibles formas de modelar este dominio en PDDL (con más o menos predicados, con más o menos tipos, con más o menos operadores...) y por ello tomaremos decisiones que vayan encaminadas a crear un modelo que no contenga ineficiencias innecesarias.

Algunas cosas que tendremos en cuenta en la solución propuesta:

- intentaremos minimizar el número de operadores y el factor de ramificación de los mismos, de forma que se reduzca la exploración de que operadores son aplicables en cada momento;
- evitaremos operadores con parámetros similares, de forma que la existencia de unos objetos u otros dirija la instanciación de operadores;
- usaremos tipos en las variables para reducir en lo posible la cantidad de objetos que el planificador comprobará para cada parámetro del operador;
- intentaremos evitar el uso de **exists** y **forall** en las precondiciones y efectos de los operadores, ya que tienen un impacto negativo en el tiempo de cómputo y, en la práctica, aumentan el factor de ramificación por la existencia de variables ocultas (variables a instanciar que no son parámetros) dentro del operador;
- será más importante que la ejecución sea eficiente, aunque la representación sea más compleja (es decir, añadiremos predicados, operadores y tipos si eso puede facilitar la labor del planificador).

Para controlar la capacidad de los vehículos un modelo discreto donde representemos los contenedores como slots asociados al vehículo es más que suficiente, y por tanto no es necesario el uso de flujos en este problema. Para comprobar que el vehículo es refrigerado cuando se ha de transportar un paquete que requiere refrigeración hay dos opciones: o modelarlo con predicados que actúen como filtro o modelarlo mediante la creación de tipos especiales. En la solución que se presenta a continuación se ha optado por usar el potente sistema de tipos de PDDL para guiar las unificaciones de variables:

```
(define (domain SmartCovid)
  (:requirements :adl :typing)
  (:types pedido lugar vehiculo - object
    pedidoNormal pedidoRefrigerado - pedido
    pGuantes pMascaras pEPI pToallasPapel pGel pJabon pDeshechos - pedidoNormal
    pPCR pSangre - pedidoRefrigerado
    vehiculoNormal vehiculoRefrigerado - vehiculo
    direccion slot - lugar)

  (:predicates
    (estacionado ?ve - vehiculo ?d - direccion)
    (dentro ?s - slot ?ve - vehiculo)
    (libre ?s - slot))
```

```

(en ?p - pedido ?l - lugar)
(destino ?p - pedido ?d - direccion)
(servido ?p - pedido)
)

(:action recoger_pedido_normal
:parameters (?pe - pedidoNormal ?ve - vehiculo ?s - slot ?d - direccion)
:precondition (and (not (servido ?pe)) (en ?pe ?d)
                  (estacionado ?ve ?d)
                  (dentro ?s ?ve) (libre ?s)
                  )
:effect (and (en ?pe ?s) (not (en ?pe ?d))
            (not (libre ?s))
            )
)

(:action recoger_pedido_refrigerado
:parameters (?pe - pedidoRefrigerado ?ve - vehiculoRefrigerado
            ?s - slot ?d - direccion
            )
:precondition (and (not (servido ?pe)) (en ?pe ?d)
                  (estacionado ?ve ?d)
                  (dentro ?s ?ve) (libre ?s)
                  )
:effect (and (en ?pe ?s) (not (en ?pe ?d))
            (not (libre ?s))
            )
)

(:action entregar_pedido
:parameters (?pe - pedido ?ve - vehiculo ?s - slot ?d - direccion)
:precondition (and (destino ?pe ?d) (en ?pe ?s) (dentro ?s ?ve)
                  (estacionado ?ve ?d)
                  )
:effect (and (en ?pe ?d) (libre ?s) (servido ?pe)
            (not (en ?pe ?s))
            )
)

(:action ir_a
:parameters (?ve - vehiculo ?ori - direccion ?des - direccion)
:precondition (estacionado ?ve ?ori)
:effect (and (estacionado ?ve ?des) (not (estacionado ?ve ?ori)))
)
)

```

En esta solución se ha creado una jerarquía de tipos que distingue entre pedidos, vehículos y los lugares en los que pueden estar pedidos y vehículos (los slots dentro del vehículo y las direcciones de hospitales, residencias, laboratorios, el parking...). La necesidad de refrigeración de los pedidos se modela con dos subtipos (pedidoNormal y pedidoRefrigerado) y la capacidad de refrigeración de los vehículos se modela con dos subtipos (vehiculoNormal vehiculoRefrigerado). Se han creado subtipos de pedidoNormal y pedidoRefrigerado que facilitan el declarar objetos de los diferentes tipos de pedidos (pGuantes, pMascaras, pEPI, pToallasPapel, pPCR, pSangre ...) y que queden automáticamente clasificados en pedidoNormal y pedidoRefrigerado. El tipo lugar nos permite usar polimorfismo en el predicado **en** (un pedido puede estar en una dirección o en un slot), pero como los vehículos no pueden estar en un slot usamos el predicado **estacionado** que solo admite direcciones como segundo parámetro.

La lista de predicados es la siguiente:

- **en**, un predicado polimórfico que nos dice en que lugar están los pedidos en cada momento. Se usa además para indicar el lugar de recogida del pedido (donde se encuentra en el estado inicial).
- **destino**, que nos dice la dirección de entrega del pedido.
- **servido**, que nos dice si un pedido ya ha sido entregado en destino. El plan acabará cuando todos los pedidos han sido servidos (que es una condición más fácil de comprobar que mirar, para todo pedido, que está en la dirección de entrega).

- **estacionado**, un predicado que nos dice en que dirección se encuentra cada vehículo en todo momento.
- **dentro**, que nos dice que un slot pertenece a un vehículo.
- **libre**, que nos dice que el slot no está ocupado por ningún pedido. Podríamos usar el predicado **en** para saber si tiene algún pedido en su interior, comprobado que para todo elemento en el modelo del problema no hay ninguno que está **en** ese slot, pero esto implicaría el uso de **forall** o de (**not exists**). Por ello es más efectivo añadir el predicado **libre**, que sabiendo un slot nos dice si está libre o no.

El modelo tiene operadores para cada paso del problema: la recogida de pedidos (**recoger_pedido_normal**, **recoger_pedido_refrigerado**), el movimiento de los vehículos de una dirección a otra (**ir_a**) y la entrega del pedido en destino (**entregar_pedido**), que son autoexplicativos. En el caso de la recogida se han hecho dos operadores: uno para pedidos refrigerados que usa los tipos **pedidoRefrigerado** **vehiculoRefrigerado** para restringir las unificaciones válidas de los parámetros, y otro para pedidos no refrigerados que permite la unificación con cualquier vehículo, sea refrigerado o no. Al hacer el filtrado con tipos no hay forma de unificar estos dos operadores en uno solo (si se modelara con predicados sería fácil definir un único operador **recoger_pedido** que en la precondition use la condición (**imply (necesitaRefrigerado ?pedido) (Refrigerado ?vehiculo)**)).

Es importante resaltar que tanto en la recogida como en la entrega de pedidos se exige ya en la precondition que el vehículo esté en la dirección de recogida o de entrega, para evitar el efecto "teleportación mágica" del pedido dentro y fuera del vehículo sin estar ambos en la misma dirección. Y es también importante remarcar la importancia de que los operadores de recogida y entrega no muevan ellos mismos al vehículo entre direcciones, ya que esto impediría a un vehículo recoger o entregar más de un pedido en la misma dirección.

Este modelo permite modelar más o menos direcciones, pedidos y más o menos contenedores disponibles en cada vehículo.

- b) Nos han proporcionado una tabla como ejemplo de las peticiones que el sistema recibe cada 20 minutos, con una serie de paquetes de material (donde el * indica que ha de transportarse refrigerado) que hay que transportar entre diferentes lugares: dos almacenes, tres hospitales, tres residencias de ancianos, un laboratorio de análisis y una incineradora. También nos dice que al inicio de ese intervalo de 20 minutos tenemos solo dos vehículos eléctricos disponibles: la *Moto1* (con capacidad para 1 paquete, sin refrigeración) y la *Van2* (con capacidad para transportar hasta 3 paquetes, tanto que requieran refrigerado como no). Ambos vehículos se encuentran aparcados en un parking municipal y, después de transportar todas las peticiones, han de volver al parking para poder recargar sus baterías eléctricas. Se nos pide que el planificador genere un plan en el que los vehículos disponibles sirvan todos los pedidos, transportando cada paquete de material del lugar de origen al de destino.

id_pedido	origen	destino	paquete (* = refrigerado)
p1	Almacen1	ResidenciaSants	Guantes
p2	Almacen1	HospitalSantPau	EPIs
p3	Almacen1	ResidenciaCollblanc	Mascaras
p4	Almacen2	HospitalVallHebron	JabonesDesinfectantes
p5	HospitalClinic	Incineradora	Deshechos
p6	Almacen1	HospitalClinic	Mascaras
p7	Almacen1	ResidenciaTresTorres	EPIs
p8	Almacen2	ResidenciaTresTorres	GelesHidroalcohólicos
p9	Almacen1	ResidenciaSants	ToallasPapel
p10	Almacen2	HospitalVallHebron	PCRs*
p11	HospitalClinic	Laboratorio	PCRs*
p12	HospitalSantPau	Laboratorio	MuestrasSangre*
p13	Almacen1	HospitalSantPau	Mascaras
p14	Almacen2	HospitalVallHebron	EPIs
p15	Almacen2	ResidenciaCollblanc	GelesHidroalcohólicos
p16	HospitalVallHebron	Laboratorio	PCRs*

Describe este problema usando PDDL. Da una breve explicación de cómo modelas el problema.

En este caso el modelado del problema está totalmente marcado por el modelado del dominio del apartado anterior. El resultado es el siguiente:

```
(define (problem SmartCovid-2vehiculos-16pedidos-9lugares)
  (:domain SmartCovid)
  (:objects p1 - pGuantes
            p3 p6 p13 - pMascaras
            p2 p7 p14 - pEPI
```

```

        p9 - pToallasPapel
        p8 p15 - pGel
        p4 - pJabon
        p5 - pDeshechos
        p10 p11 p16 - pPCR
        p12 - pSangre
        Moto1 - vehiculoNormal
        Van2 - vehiculoRefrigerado
        s11 s31 s32 s33 - slot
        parking Almacen1 Almacen2 Incineradora Laboratorio
        ResidenciaSants ResidenciaCollblanc ResidenciaTresTorres
        HospitalSantPau HospitalVallHebron HospitalClinic - direccion
    )

(:init
  (destino p1 ResidenciaSants) (en p1 Almacen1)
  (destino p2 HospitalSantPau) (en p2 Almacen1)
  (destino p3 ResidenciaCollblanc) (en p3 Almacen1)
  (destino p4 HospitalVallHebron) (en p4 Almacen2)
  (destino p5 Incineradora) (en p5 HospitalClinic)
  (destino p6 HospitalClinic) (en p6 Almacen1)
  (destino p7 ResidenciaTresTorres) (en p7 Almacen1)
  (destino p8 ResidenciaTresTorres) (en p8 Almacen2)
  (destino p9 ResidenciaSants) (en p9 Almacen1)
  (destino p10 HospitalVallHebron) (en p10 Almacen2)
  (destino p11 Laboratorio) (en p11 HospitalClinic)
  (destino p12 Laboratorio) (en p12 HospitalSantPau)
  (destino p13 HospitalSantPau) (en p13 Almacen1)
  (destino p14 HospitalVallHebron) (en p14 Almacen2)
  (destino p15 ResidenciaCollblanc) (en p15 Almacen2)
  (destino p16 Laboratorio) (en p16 HospitalVallHebron)

  (estacionado Moto1 parking)
  (estacionado Van2 parking)
  (dentro s11 Moto1) (libre s11)
  (dentro s31 Van2) (libre s31)
  (dentro s32 Van2) (libre s32)
  (dentro s33 Van2) (libre s33)
)

(:goal (and (forall (?p - pedido) (servido ?p))
             (forall (?v - vehiculo) (estacionado ?v parking))
          )
)
)

```

Se ha creado un objeto por cada pedido, dirección y vehículo. Se han creado cuatro slots que representan los contenedores que tienen los vehículos para llevar los pedidos.

El estado inicial es una fotografía de la configuración que aparece en el enunciado, indicando con el predicado **en** la dirección inicial de recogida de los pedidos y con el predicado **destino** la dirección de entrega. Con el predicado **estacionado** indicamos que los vehículos están en el parking en el estado inicial. Con el predicado **dentro** indicamos que slots corresponden a cada vehículo.

El estado objetivo usa los predicados **servido** y **estacionado** (el plan acaba cuando todos los pedidos han sido servidos en la dirección de entrega y los vehículos están estacionados en el parking). Se podría haber descrito listando los dieciséis predicados **servido** (uno por pedido) y los dos predicados **estacionado**, pero en este caso se ha optado por la expresión **forall** en adl, ya que nos permite cambiar el número de pedidos y de vehículos del problema sin tener que cambiar la descripción del estado objetivo, y no nos añade complejidad (en FastForward el **forall** en los objetivos se instancia una sola vez generando los predicados individuales, con un coste lineal respecto al número de objetos a explorar).